

Ejercicio 1:

Considere la ecuación diferencial (problema de valor inicial):

$$y' = (t+2) \cdot e^{-y}, t > 0$$

Realice un paso del método de Heun para resolver la ecuación diferencial, siendo $y(0) = 1.44$ y usando un paso de integración 0.17 . Redondee la respuesta a 3 decimales.

In [1]:

```
import numpy as np

def fun(t,y):
    return (t+2) * np.e**(-y)

y_0 = 1.44
t_0 = 0
delta_t = 0.17
N = 1

val = fun(t_0, y_0)
y_1 = y_0 + delta_t/2 * (val + fun(t_0 + delta_t, y_0 + val * delta_t))

print(np.round(y_1, 3))
```

1.521

Ejercicio 2:

Considere la función

$$f(x) = \cos(x)$$

en el intervalo $[0; 0.5]$. Utilizando interpolación cuadrática con nodos equi-espaciados en dicho intervalo, encuentre una aproximación al mínimo de la función. Redondee su respuesta a 3 decimales.

In [2]:

```
f = lambda x: np.cos(x)

a = 0
c = 0.5

# Como tenemos que buscar la sonrisa, y f(c) < f(a), duplicamos h, y ahora b no esta
h = c - a
b = c + h

ya, yb, yc = f(a), f(b), f(c)
hmin = (4 * yb - 3 * ya - yc) / (4 * yb - 2 * ya - 2 * yc) * h

print(f'Mínimo: {np.round(a + hmin, 3)}')
```

Mínimo: 0.538

Ejercicio 3:

Considere la ecuación diferencial ordinaria (problema de valor inicial)

$$y' = f(t, y), t \in (a, b)$$

$$y(a) = y_0$$

Se resuelve numéricamente usando el método Runge-Kutta 4. Cuando se divide el intervalo en 1.866 sub-intervalos iguales, se obtiene $y(b)$ igual a 0,011. Al duplicar el número de sub-intervalos, se obtiene $y(b)$ igual a 0,014. Estime el número de sub-intervalos necesarios para que el error en sea menor a 10^{-6} . La respuesta debe ser un número entero.

In [3]:

```
n = 4

N1 = 1.866 # cantidad de intervalos -> 1/delta_t
y1 = 0.011

N2 = 2*N1
y2 = 0.014

error = 1e-6

c = (y2 - y1)/((1 - 1/2**n) * (1/N1)**n) #calculo primero la constante

delta_t = (error/c)**(1/n) #el error

n_iter = 1/delta_t #las iteraciones

print(np.ceil(n_iter))
```

15.0

Ejercicio 4:

Considere la iteración punto fijo

$$y_{k+1} = e^{\frac{1}{3}} \cdot e^{-y}$$

Se sabe que converge a un punto fijo en el intervalo entre 0 y 17.429. Si se parte del punto medio de dicho intervalo, estime el número de iteraciones necesarias para alcanzar el punto fijo con un error menos a 10^{-6} .

In [33]:

```
tol = 1e-6
fun = lambda y: np.exp(1/3 * np.exp(-y))
maxiter = 10000
x = 17.429/2
xk = [x]
niter = 0

for k in range(maxiter):
    xk = np.append(xk, fun(xk[-1]))
    niter += 1
    if abs(xk[-1] - xk[-2]) <= tol: break

print(niter)
```

8

In [34]:

```
fun(xk[-1]), xk[-1], xk[-2]
```

Out[34]: (1.1154480112691452, 1.1154480660178712, 1.115447616777507)

Ejercicio 5:

Considere la ecuación

$$x^3 - a = 0$$

con $a = 2$. Usando el método de Newton-Raphson y comenzando de $x=2$, estime el número de iteraciones necesarias para alcanzar un error menor a 10^{-6} .

In [5]:

```
f = lambda x: x**3 - 2
deriv = lambda x: 3*x**2
x_ini = 2
tol = 1e-6
maxiter = 1000

iter = 0

xk = [x_ini]

for i in range(maxiter):
    iter += 1
    xk = np.append(xk, xk[-1] - f(xk[-1]) / deriv(xk[-1]))
    if abs(xk[-1] - xk[-2]) <= tol: break

print(iter)
```

5

Ejercicio 6:

Considere el sistema lineal de ecuaciones $A \cdot \overrightarrow{x} = \overrightarrow{b}$ donde:



Partiendo de



realice un paso del método de Jacobi. Dé el valor de la última componente de $\overrightarrow{x_1}$ redondeando a 3 decimales.

In [43]:

```
A = np.array([
    [4, 3, 7, 1, 5],
    [2, 4, 3, 2, 5],
    [4, 3, 1, 3, 3],
    [4, 10, 8, 9, 3],
    [3, 9, 10, 6, 10]
])

b = np.array([
    [1],
    [7],
    [1],
    [2],
    [7]
]).reshape(-1)

print((b/np.diagonal(A))[-1])
```

0.7

Ejercicio 7:

Una función toma los valores 77, 46 y 37 en los puntos 0.353, 0.854 y 0.205, respectivamente. Determine el coeficiente del término de mayor orden del polinomio interpolador que pasa por esos nodos. Redondee su respuesta a 3 decimales.

In [36]:

```
def lagran2(x, y):
    res, base = np.array([0]), np.append(False, np.full(x.size - 1, True))
    for i, out in enumerate(y):
        idx = np.roll(base, i)
        res = np.polyadd(res, np.poly(x[idx]) / np.prod(x[i] - x[idx]) * out)
    return res

x = np.array([.353, .854, .205])
y = np.array([77, 46, 37])
lagran2(x, y)[0].round(3)
```

Out[36]: -511.782

Ejercicio 8:

Considere la función

$$f(x) = \cos(x)$$

en el intervalo $[1;4]$. Determine el lado izquierdo del intervalo resultante de 1 paso del método de la razón áurea para buscar un mínimo de la función en dicho intervalo. Redondee su respuesta a 3 decimales.

In [27]:

```
def fun(x):
    return np.abs(np.sin(x))

a = 1
b = 4

r = (-1 + np.sqrt(5))/2

c = a + (1 - r) * (b - a)
d = a + r * (b - a)

# como me piden el limite izquierdo solo me importa el valor que va a tomar a
if fun(c) > fun(d): a = c

print("Limite izquierdo:", np.round(a,3))
```

Limite izquierdo: 2.146

Ejercicio 9:

Considere la matriz



¿Admite descomposición Cholesky? Justifique su respuesta.

In [28]:

```
def has_cholesky(A):
    return np.allclose(A, A.T) and np.all(np.linalg.eig(A)[0] > 0)
```

```
A = np.array([
    [14, 38, 54],
    [38, 10, 162],
    [54, 162, 243]
])

print(f'Tiene descomposición Cholesky: {"Sí" if has_cholesky(A) else "No"}')
```

Tiene descomposición Cholesky: No

Ejercicio 10:

Considere el número en punto flotante IEEE 754 de 64 bits

0010 0001 0000 0011 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000

Determine la amtisa en decimal (en caso de ser normal, sumar 1).

In [42]:

```
num = '00100001000000110000000000000000000000000000000000000000000000'
mant = int(num[12:], 2) / 2**len(num[12:]) + 1 # Es normal.
print(mant)
```

1.1875